

Machine learning (ML) is a subfield of artificial intelligence that focuses on the development of algorithms and statistical models that allow computer systems to **learn from data** and improve their performance on a specific task without being explicitly programmed.

Types of Machine Learning

There are three main types of machine learning, categorized by how the algorithm learns from the data.

1. Supervised Learning 🤖

In supervised learning, the model is trained on a **labeled dataset**, which means the training data includes both the input features and the corresponding correct output (the "label"). The model's goal is to learn the mapping function from the input to the output.

- **How it works:** Think of it like a student learning with a teacher. The teacher (the labeled data) provides the correct answers for a set of questions. The student (the model) learns to solve the problems by understanding the relationship between the questions and the answers.
- **Common tasks:**
 - **Classification:** Predicting a discrete category, like "spam" or "not spam" for an email.
 - **Regression:** Predicting a continuous value, like the price of a house.
- **Examples:** Spam filters, image recognition (e.g., identifying a dog in a picture), predicting stock prices.

A. Regression

Predicting a continuous output value.

- **Linear Regression**
 - Simple Linear Regression (SLR)
 - Multiple Linear Regression (MLR)
- **Polynomial Regression**
- **Ridge Regression**
- **Lasso Regression**
- **Non-linear Regression**
- **Time Series Forecasting** (Often uses regression techniques)
- **Other (OLR)** - *(Assuming this refers to Ordinary Least Squares, a common method for linear regression)*

B. Classification

Predicting a discrete category or class label.

- **Logistic Regression**
 - **K-Nearest Neighbors (KNN)**
 - **Decision Tree (DT)**
 - **Random Forest (RF)**
 - **Naive Bayes**
 - **Support Vector Machine (SVM)**
 - **Artificial Neural Networks (ANN)**
 - **Gradient Boosting Machines (GBM)**
 - **Extreme Gradient Boosting (XGBoost)**
 - **CatBoost**
 - **AdaBoost**
-

2. Unsupervised Learning 🧐

In unsupervised learning, the model is given an **unlabeled dataset**, meaning there are no correct outputs provided. The model must discover hidden patterns and structures in the data on its own.

- **How it works:** This is like a student learning by exploring. Without a teacher, the student looks at a collection of items and groups them based on their similarities.
 - **Common tasks:**
 - **Clustering:** Grouping similar data points together.
 - **Association:** Discovering rules that describe relationships between variables.
 - **Dimensionality Reduction:** Reducing the number of features in a dataset while retaining important information.
 - **Examples:** Customer segmentation for marketing (grouping customers by behavior), market basket analysis (finding items that are often bought together, e.g., "people who buy bread also buy milk"), and anomaly detection.
-

3. Reinforcement Learning 🎮

Reinforcement learning involves an "agent" that learns to make decisions by performing actions in an "environment" to maximize a cumulative "reward." The agent learns through trial and error, similar to how a person learns to play a video game.

Instead of being trained on a labeled dataset, the agent learns through a process of **trial and error**. It performs an **action** in a given **state**, and the environment provides a positive **reward** for a good action or a negative one for a bad action. The agent uses this feedback to adjust its policy, gradually learning which actions lead to the best outcomes. This is similar to how a human or an animal learns a new skill, like playing a game or riding a bike.

- **How it works:** The agent receives a positive reward for a desired action and a penalty for an undesired one. Over time, the agent learns the best strategy to achieve its goal.
 - **Common tasks:**
 - Training robots to perform tasks.
 - Developing systems to play complex games.
 - Optimizing resource management.
 - **Examples:** Self-driving cars (rewarded for staying on the road and reaching the destination), game AI (e.g., AlphaGo), and robotics.
-

Data scaling and normalization

Data scaling and normalization are data preprocessing techniques used to standardize the range of independent numerical features. The goal is to transform the data so all features are on a similar scale. This prevents features with naturally large values (like salary) from dominating those with smaller values (like age), which can bias the model.

Data scaling and normalization are crucial preprocessing steps that standardize the range of features to ensure fair and efficient model training.

- **Improved Performance:** Algorithms that use **distance metrics** (like KNN and SVM) are heavily influenced by the magnitude of features. Scaling ensures that all features contribute equally to the distance calculation, preventing features with a larger range from dominating the result.
- **Faster Convergence:** For algorithms based on **gradient descent** (like neural networks), scaling helps them find the optimal solution much more quickly. Without scaling, the optimization process can be slow and inefficient.
- **Equal Contribution:** Scaling guarantees that no single feature is given undue importance just because it has a larger numerical value. This leads to a more robust and accurate model, free from unintentional **bias**.

- **Avoidance of Bias:** Without normalization, a model might incorrectly weigh a feature with a larger magnitude as more important, even if its actual predictive power is low.

1. Min-Max Normalization

Min-Max Normalization, often simply called "normalization," is a popular method that rescales a feature to a fixed range, typically **[0, 1]**. This technique is especially useful for algorithms that are sensitive to the magnitude of features, such as K-Nearest Neighbors (KNN) and Support Vector Machines (SVM), as well as neural networks.

The core idea is to transform the data so that the minimum value of a feature becomes 0 and the maximum value becomes 1. All other values are scaled linearly in between.

The formula is:

$$X_{normalized} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

2. Standardization (Z-score normalization)

This technique transforms the data to have a **mean of 0 and a standard deviation of 1**. It's preferred when the data follows a Gaussian distribution or when algorithms assume a normal distribution. Unlike Min-Max, standardization doesn't bound the data to a specific range, so it's less affected by outliers.

3. Robust data scaling:

It is a method for scaling features that is **resilient to outliers**. Unlike standard scaling methods that use the mean and standard deviation (which are easily skewed by outliers), robust scaling uses the **median** and **interquartile range (IQR)**. These statistics are not affected by extreme values, so the scaling process accurately represents the majority of the data without being distorted by a few outliers

Handling Outliers

Handling outliers is a critical step in data preprocessing. Outliers are data points that deviate significantly from other observations. They can negatively impact the performance of many machine learning models.

Scaling Categorical Data:

In machine learning, "scaling" is a term almost exclusively used for numerical data. For categorical variables, the correct term is **encoding**. You're converting categories into a numerical format that a model can understand. Here are notes on the most common encoding methods and related terms.

What is Encoding?

Encoding is the process of converting categorical data (like "red," "medium," "New York") into a numerical format. This is necessary because most machine learning algorithms require all input features to be numerical. The choice of encoding method depends on whether the categories have an inherent order.

One-Hot Encoding: Converts **nominal** (unordered) categorical data into a binary format. It creates a new column for each category, assigning a 1 where the category is present and 0 everywhere else.

- **Example:** A color feature with values 'Red' and 'Blue' becomes two new columns, color_Red and color_Blue. A row with 'Red' becomes [1, 0].

Label Encoding: Converts **ordinal** (ordered) categorical data into integer values. It assigns a unique integer to each category based on its rank.

- **Example:** A size feature with values 'Small', 'Medium', and 'Large' becomes 0, 1, and 2 respectively, which maintains the logical order.

Target Encoding: A more advanced technique that replaces a categorical value with the **mean of the target variable** for that category. It's used when there are many unique categories.

- **Example:** In a model predicting house prices, you could replace the city feature with the average house price for each city.
-

Data Distribution:

In statistics and data analysis, **data distribution** describes how data points are spread out across a range of values. The shape of this spread is key to understanding your data and choosing the right machine learning model.

Normal Distribution

A **normal distribution**, also known as a Gaussian distribution or bell curve, is the most common and important type of data distribution. In a normal distribution, most data points cluster around the **mean**, and the further a data point is from the mean, the less likely it is to occur.

- **Key Characteristics:** It is symmetrical, with the mean, median, and mode all being equal and located at the center.

Skewed Distribution

A **skewed distribution** is one where the data points are not symmetrical and are concentrated on one side of the distribution. This is a common occurrence in real-world datasets.

- **Positive Skew (Right Skew):** The long tail of the distribution points to the right. This means most of the data is concentrated on the left side, and there are a few very large values (outliers) pulling the mean to the right.
 - **Example:** Income data. Most people earn a moderate income, but a small number of people have extremely high incomes.
- **Negative Skew (Left Skew):** The long tail of the distribution points to the left. Most of the data is concentrated on the right side, and there are a few very small values pulling the mean to the left.
 - **Example:** Exam scores. Most students score high marks, but a few students perform poorly.

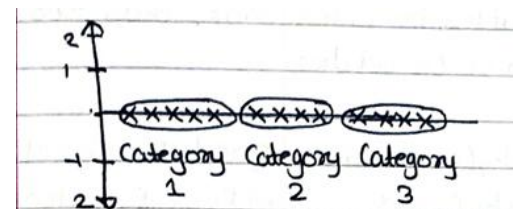
Uniform Distribution ☐

A **uniform distribution** is one where every value within a given range has an equal probability of occurring. The data is spread evenly across the range.

- **Key Characteristics:** It appears as a flat, horizontal line on a histogram.
- **Example:** The outcome of rolling a fair die. Each number from 1 to 6 has an equal $1/6$ probability of being rolled.

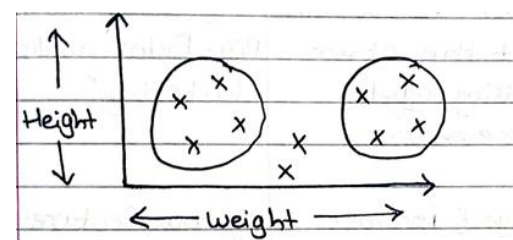
Univariate Graphs 🇮🇹

Univariate graphs visualize the distribution of a **single variable**. They help you understand its central tendency, spread, and shape. Example: box plot, histogram.



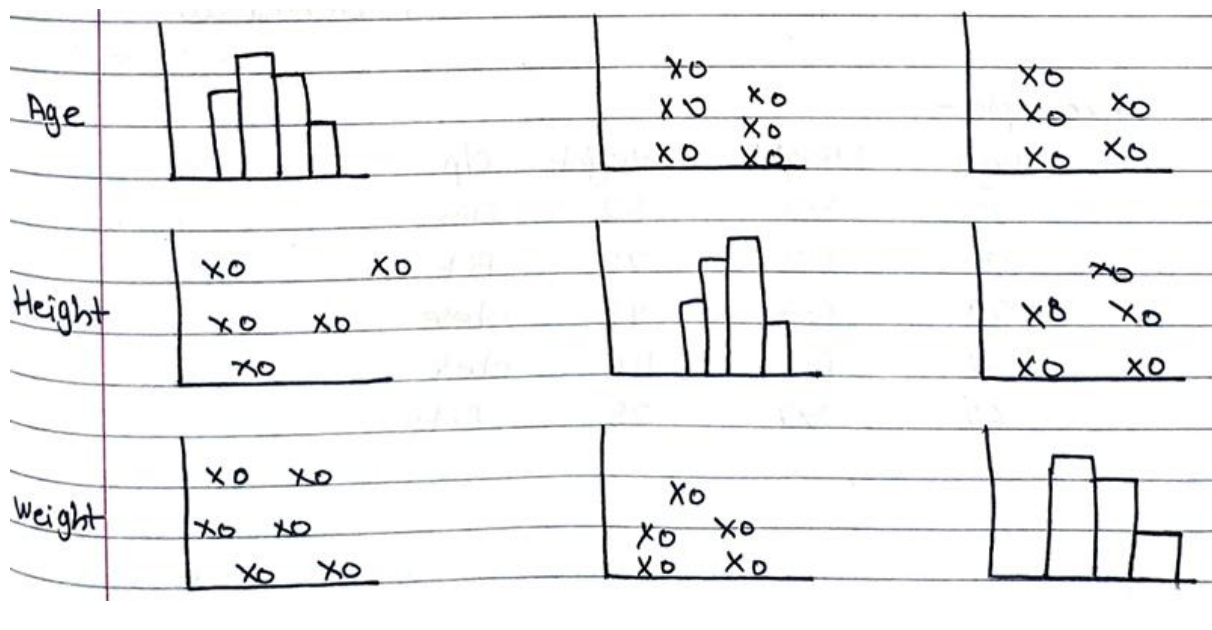
Bivariate Graphs 📊

Bivariate graphs show the relationship between **two variables**. They're used to discover correlations or dependencies between features. Example: Scatter, line Plot.



Multivariate Graphs 🌐

Multivariate graphs display the relationships among **three or more variables** at once. They're essential for more complex analysis where you need to see how multiple factors interact. Example: Bubble Chart, Pair Plot.



Data discretization:

often called **binning**, is the process of converting continuous numerical data into discrete, categorical bins or intervals. This is done to simplify the data, reduce the effects of minor observation errors, and can be useful for algorithms that handle categorical data better.

Types of Discretization

- **Equal Width Binning:** Divides the data range into bins of **equal width**. Simple but can be problematic for skewed data.
 - *Example:* For data from 0-100, bins could be [0-20], [20-40], etc.
- **Equal Frequency Binning:** Divides data into bins containing an **equal number of data points**. Ensures all bins are populated but can lead to uneven bin widths.
 - *Example:* A dataset of 100 values could be split into four bins, each with 25 data points.
- **Clustering-Based Binning:** Uses a clustering algorithm (like K-Means) to define bin boundaries based on **natural data groupings**.
- **Decision Tree-Based Binning:** A supervised method that uses a decision tree to find optimal bin boundaries that **best separate the target variable**.
- **Custom Binning:** Bins are defined **manually** based on specific business rules or domain knowledge.
 - *Example:* Tiers for loan amounts (e.g., \$1-\$10,000, \$10,001-\$50,000).
- **Entropy-Based Binning:** A supervised method that uses **information gain** to find bin boundaries that maximize class separation.

- **Frequency-Based Binning:** A general term often used interchangeably with **Equal Frequency Binning**.
-

Data Preprocessing:

Data preprocessing is the process of cleaning and transforming raw data into a format that's suitable for machine learning models. It's a critical step that ensures your model is trained on quality data, leading to more accurate results.

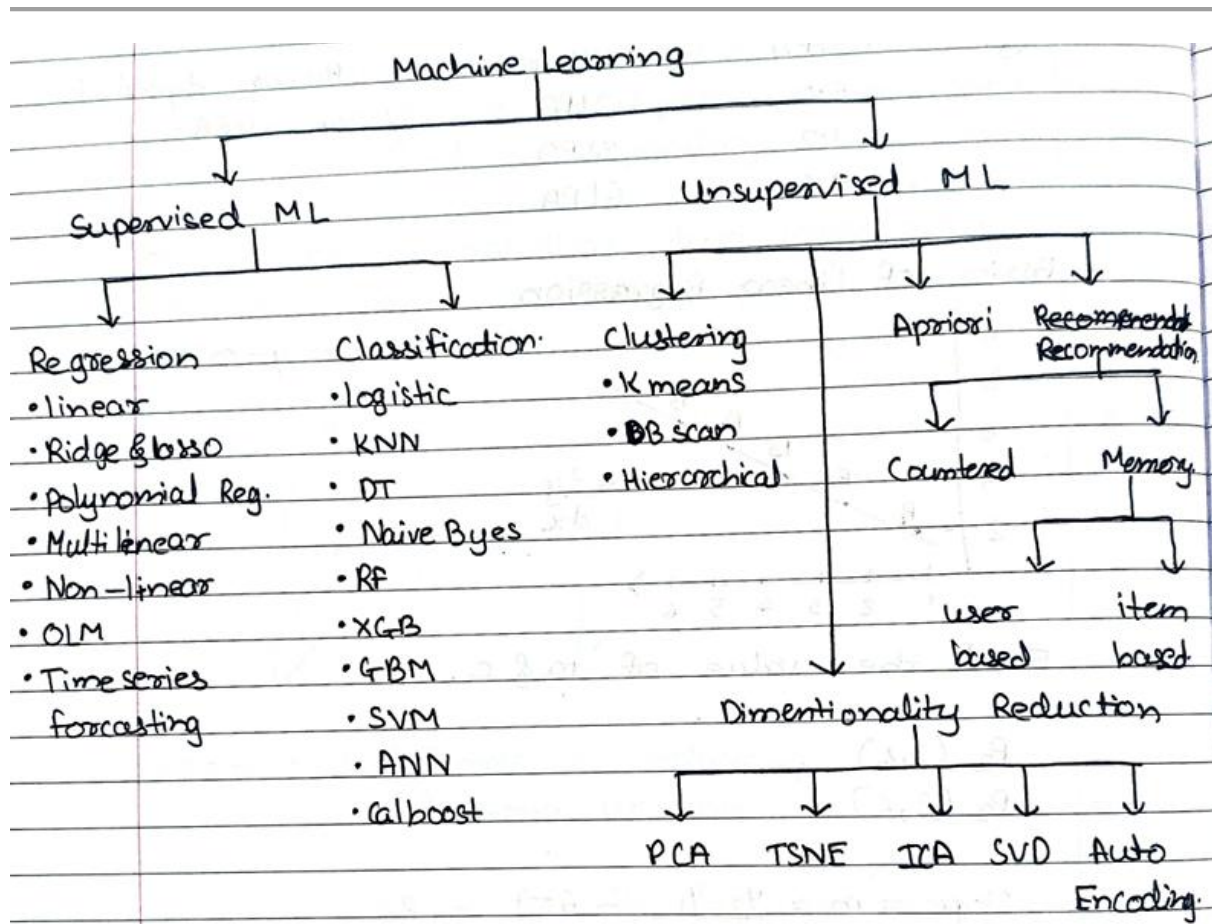
Here are the key techniques:

- **Handling Missing Values:** This involves dealing with empty data points. You can either **impute** them by filling in the blanks with a calculated value (like the mean or median of the feature) or **delete** the rows or columns with too many missing values.
- **Handling Outliers:** Outliers are data points that are significantly different from the rest. You can handle them by **deleting** them, **capping** them at a certain maximum or minimum value, or using a **robust scaling** method that is not affected by their presence.
- **Normalization/Scaling:** This technique standardizes the range of numerical features. It ensures that features with larger values don't dominate those with smaller ones. Common methods include **Min-Max scaling** (to a range like 0-1) and **Standardization** (to a mean of 0 and standard deviation of 1).
- **Encoding Categorical Data:** Most machine learning algorithms require numerical input. Encoding converts non-numerical categories into a numerical format.
 - **One-Hot Encoding** is used for categories without an order (e.g., 'city').
 - **Label Encoding** is used for categories with a clear order (e.g., 'small', 'medium', 'large').
- **Feature Extraction:** This is the process of creating new features from the existing ones. The goal is to reduce the dimensionality of the dataset while preserving important information. **Principal Component Analysis (PCA)** is a popular method for this.
- **Data Splitting:** This final step divides the dataset into at least two parts: a **training set** (used to train the model) and a **test set** (used to evaluate the model's performance on unseen data). A common split ratio is 80% for training and 20% for testing.

KNN imputation: It is a method for filling in missing data by using information from the **k-nearest neighbors**.

Here's a simpler way to think about it:

- For a data point with a missing value, the algorithm finds the k data points that are most similar to it.
- It then fills in the missing value with the average (for numerical data) or the most common value (for categorical data) from those k neighbors.



Simple Linear Regression:

Simple Linear Regression (SLR) is a statistical method used to model the relationship between two variables. It assumes a linear relationship, meaning the two variables have a straight-line connection.

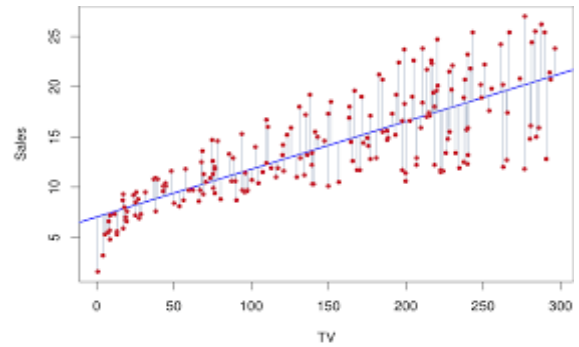
The core idea is to find the "best-fit" straight line that represents the relationship between:

- **The Independent Variable (X):** The predictor or feature used to make a prediction.
- **The Dependent Variable (Y):** The response or target variable that you are trying to predict.

The equation for this line is:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

- Y: The predicted value of the dependent variable.
- X: The value of the independent variable.
- β_0 : The **intercept**, where the line crosses the Y-axis.
- β_1 : The **slope**, which indicates how much Y changes for every one-unit change in X.
- ϵ : The **error term**, which represents the noise or residual difference between the actual and predicted values.



Example: SLR can be used to predict a student's exam score (Y) based on the number of hours they studied (X). The model would find a line that best fits the data points, allowing you to estimate a student's likely score given their study time.

Multilinear Regression:

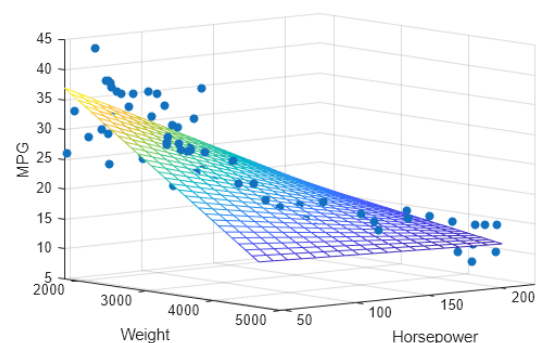
Multiple Linear Regression (MLR) is a statistical method used to model the relationship between a single **dependent variable** and **two or more independent variables**. It's an extension of simple linear regression, which only uses one independent variable.

The goal of MLR is to find the "best-fit" linear equation that describes how the dependent variable changes in response to changes in all the independent variables. This creates a multi-dimensional line, or **hyperplane**, that fits the data.

The equation for multiple linear regression is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

- Y: The predicted value of the dependent variable.
- $X_1, X_2, \dots, X_n$: The values of the independent variables.
- β_0 : The **intercept**, where the hyperplane crosses the Y-axis.
- $\beta_1, \beta_2, \dots, \beta_n$: The **coefficients** for each independent variable, representing the change in Y for a one-unit change in that specific X variable, assuming all other variables are held constant.
- ϵ : The **error term**.



Example: MLR can be used to predict a house's price (Y) based on multiple factors like its square footage (X_1), the number of bedrooms (X_2), and the distance from the city center (X_3).

). This allows for a more comprehensive and accurate prediction than using just one factor alone.

Regression Working

Regression is a supervised learning task used to predict a continuous value. It works by finding the **best-fit line** (or hyperplane) that represents the relationship between a dependent variable and one or more independent variables. The model's goal is to make the predictions as close as possible to the actual data points. It does this by minimizing the difference between the predicted values and the actual values. This difference is measured by a **cost function**.

Core Concepts

- **Independent Variable(s) (X):** These are the input features used to make a prediction. For example, in a model predicting house prices, features like square footage, number of bedrooms, and location would be the independent variables.
- **Dependent Variable (Y):** This is the target variable you are trying to predict. In the house price example, the price of the house itself is the dependent variable.
- **Coefficients (β):** These are the weights assigned to each independent variable. They determine the slope and intercept of the best-fit line. The model learns these coefficients during training.

Cost Function: A function that measures the model's performance by calculating the total error of its predictions. For regression, the most common cost function is **Mean Squared Error (MSE)**, which averages the squared differences between the predicted and actual values.

The most widely used is the **Mean Squared Error (MSE)**:

$$J(\beta_0, \beta_1, \dots, \beta_n) = \frac{1}{m} \sum_{i=1}^m (\hat{Y}_i - Y_i)^2$$

Where:

- m = number of training samples
- Y_i = actual value
- $\hat{Y}_i$ = predicted value from regression model
- β = regression parameters (intercept & slopes)

Gradient Descent

The most common **optimization algorithm** used to find the coefficients that minimize the cost function. It works by iteratively adjusting the coefficients in the direction of the steepest decrease in the error.

Think of it as a person trying to get down a hill in a thick fog. They can't see the bottom, but they know which way is "downhill" by feeling the slope beneath their feet. Gradient Descent works in a similar way:

- It starts with a set of random parameters.
- It calculates the **gradient** (the slope) of the cost function, which indicates the direction of the steepest ascent (uphill).
- It then takes a step in the opposite direction (downhill), moving toward the minimum error.
- This process is repeated iteratively until the algorithm reaches the bottom of the "hill," where the error is minimized.

Learning Rate

The **learning rate** is a hyperparameter that controls how big of a step the gradient descent algorithm takes to adjust the model's coefficients. Think of it as the step size when walking down a hill (the cost function).

- **If the learning rate is too high**, the model might take huge steps and overshoot the lowest point (the optimal solution), causing the model to never converge.
- **If the learning rate is too low**, the model will take tiny steps. While it will eventually reach the lowest point, it will take a very long time to train.

Finding the right learning rate is crucial for training a model effectively and efficiently.

Ridge Regression (L2 Regularization)

Ridge Regression is a model tuning method used to analyze data that suffers from multicollinearity. It applies **L2 regularization**, which adds a penalty equal to the square of the magnitude of the model's coefficients.

- **Main Goal:** To reduce overfitting by shrinking the coefficients toward zero.
- **Key Action:** It makes the coefficients smaller but **does not reduce them to exactly zero**. Therefore, Ridge Regression does not perform feature selection.

$$Cost = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Lasso Regression (L1 Regularization)

Lasso (Least Absolute Shrinkage and Selection Operator) is similar to Ridge but uses **L1 regularization**. The penalty is based on the absolute value of the coefficients.

- **Main Goal:** To reduce overfitting and perform **feature selection**.
- **Key Action:** It can shrink some coefficients all the way to zero, effectively removing those features from the model. This is why it's used for feature selection.

$$Cost = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Elastic Net (L1 + L2)

Elastic Net is a **regularized regression model** that combines the penalties of both **Lasso (L1)** and **Ridge (L2)** regression. Its purpose is to get the best of both worlds:

- It performs **feature selection** by shrinking some coefficients to zero (like Lasso).
- It handles **correlated features** effectively by keeping all of them in the model (like Ridge).

$$Cost = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left[\alpha \sum_{j=1}^p |\beta_j| + (1 - \alpha) \sum_{j=1}^p \beta_j^2 \right]$$

Ridge Regression majorly focuses on **regularization** to prevent overfitting. It makes coefficients smaller but doesn't set them to zero.

Lasso Regression focuses on both regularization and **feature selection**. It can set some coefficients to exactly zero, effectively removing those features from the model.

Elastic Net is a combination of both, providing the benefits of both regularization and feature selection.

Therefore if my job is only **feature selection** -> go for **Lasso**. If **regularization** -> **Ridge**
for both -> **Elastic net**

Underfitting

Underfitting occurs when a model is **too simple** to capture the underlying patterns in the training data. As the graph shows, this happens in the "**Underfitting Zone**," where the model's complexity is too low.

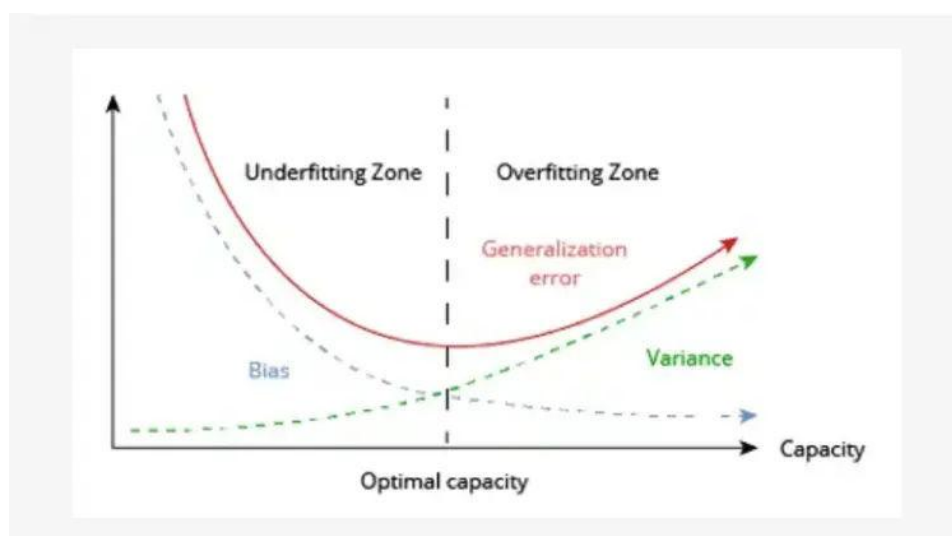
- **Result:** The model performs poorly on both the training data and new, unseen data.
- **Cause:** It has a **high bias**—it makes strong, incorrect assumptions about the data.
- **Analogy:** A student who barely studies for an exam. They don't have enough information to understand the concepts, so they perform poorly on the test.

Overfitting

Overfitting occurs when a model is **too complex** and has essentially memorized the training data, including its noise and random fluctuations. This happens in the "**Overfitting Zone**," as shown on the right side of the graph.

- **Result:** The model performs exceptionally well on the training data (very low error) but performs poorly on new, unseen data (high generalization error).
- **Cause:** It has a **high variance**—it's too sensitive to the specific details of the training data.
- **Analogy:** A student who memorizes every answer from a practice test but can't solve a new problem on the actual exam. They've learned the test, not the concepts.

The goal in machine learning is to find the "**Optimal capacity**" where the model achieves the best balance between bias and variance, resulting in the lowest possible generalization error.



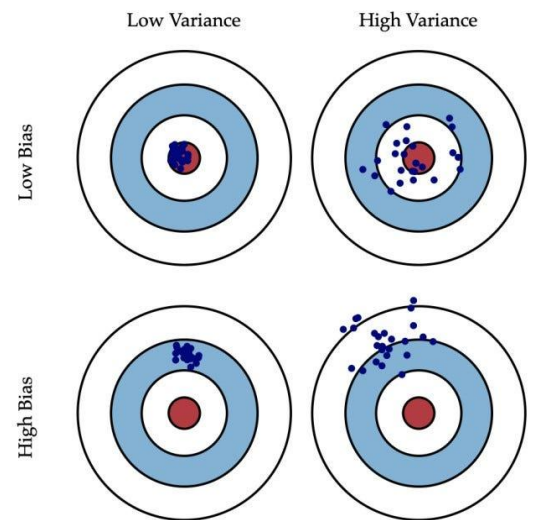
Bias-Variance Tradeoff

Bias

Bias is the error that results from a model making overly simplistic assumptions about the data. It's the difference between the model's predicted values and the actual, correct values.

When a model has **high bias**, it is **underfitted**. This means the model is too simple to capture the underlying patterns, leading to consistently large errors on both the training data and new, unseen data.

👉 Example: Using a **straight line** to fit data that is actually quadratic → high bias.

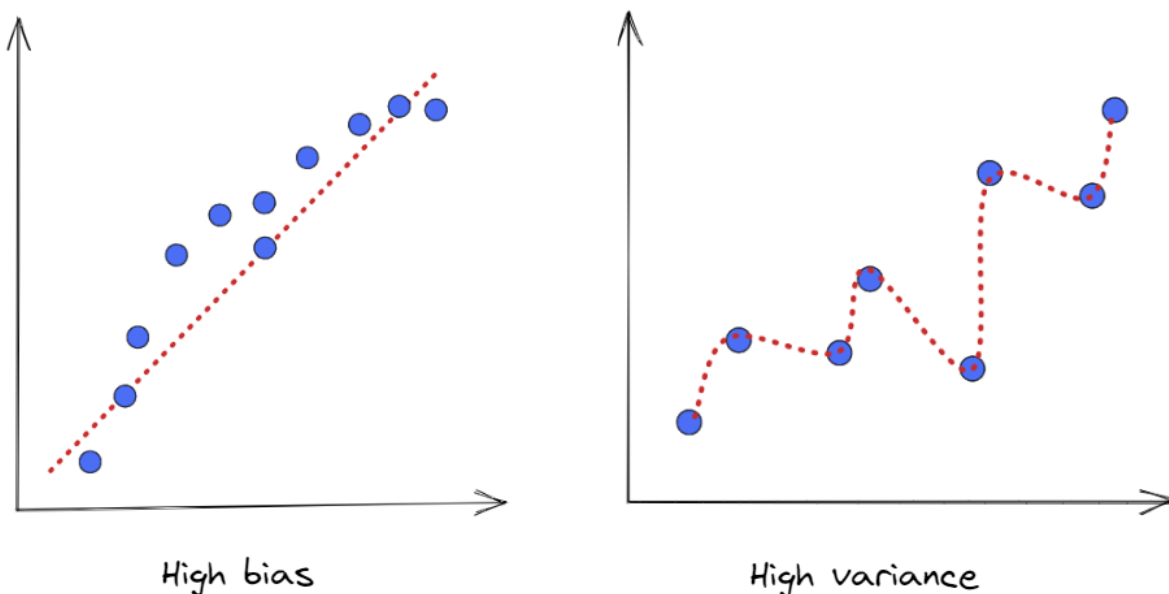


Variance

Variance is the variability of a model's predictions for a given data point. It measures how sensitive a model is to small fluctuations in the training data.

A model with **high variance** is very complex and is **overfitted**. It fits the training data extremely well, essentially memorizing the data points and their noise. As a result, it performs poorly on new, unseen test data, leading to a high test error.

👉 Example: Using a **10th-degree polynomial** to fit data that is mostly linear → high variance.



Performance evaluation is the process of measuring how well a machine learning model performs. Its main goal is to assess the model's ability to **generalize**—that is, how accurately it makes predictions on new, unseen data.

Mean Absolute Error (MAE) 🎯

MAE is the average of the absolute differences between the model's predicted values and the actual values. It gives a straightforward measure of the average error.

- **Formula:**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Mean Squared Error (MSE) 📊

MSE is the average of the squared differences between predicted and actual values. It's a common loss function used to train regression models.

- **Formula:**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) 📊

RMSE is the square root of the MSE. It's a very popular metric because it puts the error back into the same units as the target variable.

- **Formula:**

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Coefficient of Determination (R2) ✨

Also known as R-squared, this metric provides a measure of how well the independent variables explain the variance of the dependent variable. It's a score between 0 and 1.

- **Formula:**

$$R^2 = 1 - \frac{SSR}{SST}$$

- SSR (Sum of Squared Residuals) = $\sum (y_i - \hat{y}_i)^2$
- SST (Total Sum of Squares) = $\sum (y_i - \bar{y})^2$

Apriori Algorithm

The **Apriori algorithm** is a classic algorithm used for **association rule learning**. Its main goal is to discover **frequent itemsets** (groups of items that often appear together) in a transactional dataset and then generate **association rules** from those itemsets. The most common use case is **market basket analysis**, where you find what items customers tend to buy together (e.g., "people who buy bread and milk also tend to buy eggs").

Itemset → Group of items (e.g., {Milk, Bread}).

Support → How frequently an itemset appears in the dataset.

$$\text{Support}(A) = \frac{\text{Total transactions containing } A}{\text{Total Transactions}}$$

Confidence → How often item Y is bought when item X is bought.

$$\text{Confidence}(X \Rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Lift → How much more likely Y is bought when X is bought, compared to random chance.

$$\text{Lift}(X \Rightarrow Y) = \frac{\text{Confidence}(X \Rightarrow Y)}{\text{Support}(Y)}$$

- Lift > 1 → X and Y occur together more often than random (interesting).
- Lift = 1 → independent.
- Lift < 1 → negatively correlated.

Support Vector Machine

A **Support Vector Machine (SVM)** is a powerful supervised machine learning algorithm used primarily for **classification** and **regression**. Its main objective is to find the optimal decision boundary that best separates data points belonging to different classes.

How SVM Works (for Classification)

The core idea behind SVM for classification is to find a **hyperplane** that separates the data. In a 2-D space, a hyperplane is a simple line. In a 3-D space, it's a plane, and so on.

The unique aspect of SVM is that it doesn't just find any line; it finds the one that maximizes the **margin** between the two classes.

- **Margin:** The distance between the hyperplane and the closest data point from each class.

- **Support Vectors:** The data points that are closest to the hyperplane. These are the critical data points that define the position and orientation of the hyperplane.

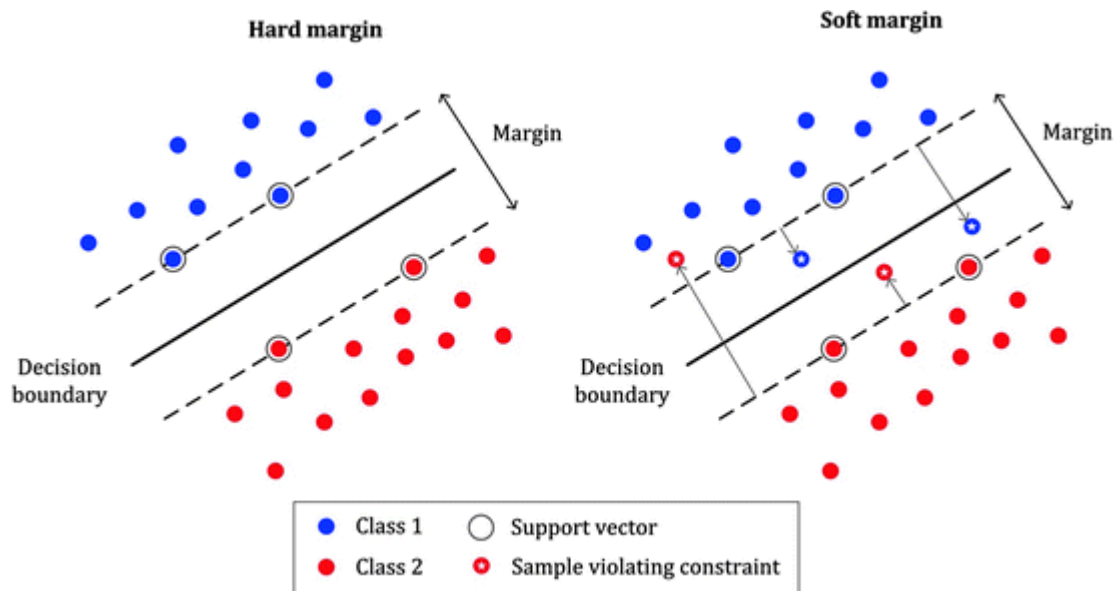
By maximizing this margin, the SVM aims to achieve the most confident separation possible, which helps the model generalize better to new, unseen data.

Hard Margin

A **hard margin SVM** is a very strict classifier. It works on the assumption that the data is **perfectly linearly separable**, meaning there is a clear boundary that can separate the two classes without any data points from one class crossing into the other's territory.

Soft Margin

A **soft margin SVM** is a more flexible and practical classifier. It allows for a small number of misclassifications or outliers within the margin or on the wrong side of the hyperplane.



SVM Numerical:

Qus: Consider the following set of data points given, find the optimal hyperplane for these data points.

(1, 0.5), (1, 1), (1, -0.5), (-0.5, 0.5), (0.5, 0.5), (2, 0), (4, 0), (4.5, 1), (4.5, .5), (5, -1), (5.5, 0)

Solution:

Step1: Draw all the points on plane.

Step2: Find the support vectors.

Step3: Add the bias(b) value in vectors to convert 2D to 3D form.

Step4: Consider Parameters for every support vectors like α_1, α_2 etc.

Step5: Find the linear equations and solve then to find parameter values.

Step6: Final equation for hyperplane is

$$\bar{w} = \sum_{i=1}^n \alpha_i \cdot S_i$$